数据结构与算法基础

2026-2027年度秋季学期

通信学院 马然



教学安排

◇ 教材：《数据结构》严蔚敏，清华大学出版社

◇ 参考书：涉及到数据结构、算法分析的书籍

◇ 课堂要求：

认真记笔记，遵守课堂纪律

按时交电子版作业（每章结束后）

准时上课（不迟到，校规：缺勤达1/3、取消考试资格）

◇ 答疑安排：课前课后、学习通、QQ群



QQ群

实名：学号-姓名

关于课程的所有问题在群里问，勿私聊！！！

第1章 绪论

1.1 什么是数据结构与算法

1.2 数据结构的基本概念和术语

1.3 抽象数据类型的表示与实现

1.4 算法和算法分析

1.1 什么是数据结构与算法

◇数据结构与算法的关系

数据结构+算法=程序

◇程序设计的实质

数据结构的设计——数据表示

算法设计——数据处理

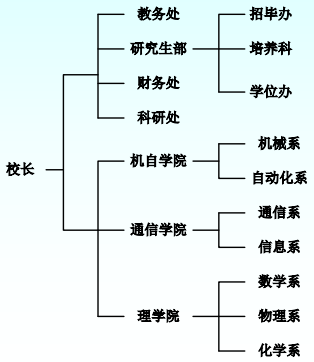
（1） 学习数据结构的意义

例1 某班学生的成绩信息处理

学号	姓名	高数	英语	电路	……
09001	李红	88	76	76	
09002	周力维	76	80	79	
09003	乔初	93	81	65	
09004	张玉	65	92	81	
09005	林晓晓	82	63	83	
09006	刘有利	77	89	86	
⋮	⋮	⋮	⋮	⋮	⋮

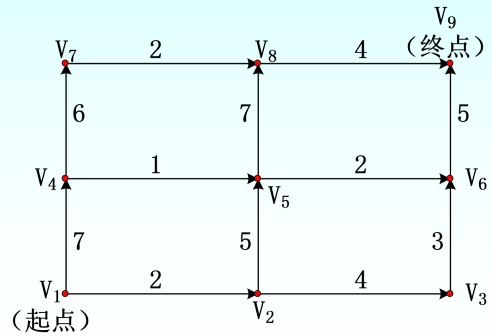
（1）学习数据结构的意义

例2 某高校的组织、管理



（1）学习数据结构的意义

例3 网络的设计



（1）学习数据结构的意义

从以上的例子可以看到：

- 首先描述这类问题的数学模型不再只是数值方程，而是诸如表、树和图等非数值性数据。

  如何表示这些数据？如何存储这些数据？

- 其次，求解这类问题不再只是数值计算，而是要：
  - 对一些信息表进行插入、删除、排序、查找；
  - 对树进行遍历；
  - 对图作较为复杂的非数值计算等。

  如何对这些数据进行操作（或运算）？

（2） 学习算法的意义

- 100元钱买100支笔(钢笔3元一支，圆珠笔2元一支，铅笔0.5元一支)。问：有哪些购买方案？

解法1：x、y、z分别表示钢笔、圆珠笔、铅笔的个数

```
for (x=0; x<=100; x++)
  for (y=0; y<=100; y++)
    for (z=0; z<=100; z++)
      if((x+y+z==100)&&(3*x+2*y+0.5*z==100))
        printf("%d,%d,%d",x,y,z);
```

- 用三层循环：内循环执行了100万余次

$$\sum_{X=0}^{100}\sum_{Y=0}^{100}\sum_{Z=0}^{100}1=101\times101\times101=1030301$$

- 结果：在某机器上运行用了50分钟。

（2）学习算法的意义

- 100元钱买100支笔(钢笔3元一支，圆珠笔2元一支，铅笔0.5元一支)。问：有哪些购买方案？

解法2：

经过分析，钢笔最多买20支，圆珠笔最多买34支。

```
for (x=0;x<=20;x++)
  for (y=0;y<=34-x;y++)
    if (3*x+2*y+0.5*(100-x-y)==100)
      printf("%d,%d,%d",x,y,100-x-y);
```

- 优化后，只需要二层循环：525次

$$\sum_{X=0}^{20}\sum_{Y=0}^{34-X}1=\sum_{X=0}^{20}\underbrace{(1+1+1\cdots+1)}_{34-X+1}=525$$

- 结果：只运行了2秒，相差1500倍。

（2） 学习算法的意义

- 100元钱买100支笔(钢笔3元一支，圆珠笔2元一支，铅笔0.5元一支)。问：有哪些购买方案？

解法3：

0≤x≤20；x、y、z是整数；且满足如下的方程组：

$$\begin{cases} x+y+z=100 \\ 3x+2y+0.5z=100 \end{cases}$$

$$\begin{cases} y=\dfrac{100-5x}{3} \\ z=100-x-y \end{cases}$$

思考：
需要几层循环？执行多少次？
试写出对应的算法！

（2） 学习算法的意义

➢学习和研究算法可以明确分析所得到的算法的好坏，寻找能满足要求的较优算法，从而更加高效地解决问题。

➢学习算法设计的方法和算法分析的技术后，可以帮助我们设计较好的算法，分析算法的优、缺点，从而找出解决某一问题的最好方法。

1.2 数据结构的基本概念和术语

数据结构的三要素：

- 数据的**逻辑结构**
- 数据的**物理结构**(或**存储结构**)
- 对**数据的运算**(或**操作**)。

基本概念和相关术语

- ***数据***：数据是信息的载体，是描述客观事物的数字、字符、以及所有能输入到计算机中，被计算机程序识别和处理的符号的集合。
  - 数值性数据
  - 非数值性数据
- ***数据元素***：是数据的基本单位，有时一个数据元素由数据项组成（具有独立含义的最小标识单位）。
- ***数据对象***：数据的子集。具有相同性质的数据元素的集合。
- ***数据结构***：同一数据对象中的各数据元素之间存在的关系。

数据结构的图例

学生成绩登记表

	学号	姓名	班级	成绩
1	900425	张勇	电02	85
2	910225	李军	建13	90
3	930443	赵红	计31	92
4	940112	刘明	制42	80
:	:	:	:	:

数据对象　　数据项　　数据元素

逻辑结构

➢***逻辑结构的定义***：

数据间关系的描述

➢***4种基本逻辑结构***：

（1）集合结构：属于同一个集合

（2）线性结构：一对一

（3）树形结构：一对多

（4）图形结构：多对多

逻辑结构的二元组表示

Data_Structure = { D, R }

其中　D：数据元素的有限集，$D=\{d_i|1\le i\le n,n\ge 0\}$；

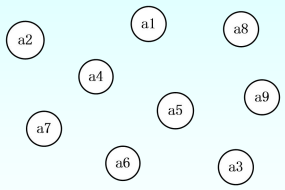
d_i表示第i个数据元素；n为数据元素的个数；

特别，n＝0，D为空集，则无结构可言。

R：D上关系的有限集，$R=\{r_j|1\le j\le m,m\ge 0\}$；

r_j表示第j个二元关系，m为D上关系的个数。

逻辑结构——集合结构



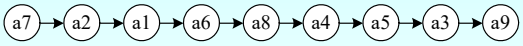
“属于同一个集合”

Pointset＝（D，R），其中：

D＝{a1，a2，a3，a4，a5，a6，a7，a8，a9}

R＝Ø

逻辑结构——线性结构



“一对一”

Linearity＝（D，R），其中：

D＝{a1，a2，a3，a4，a5，a6，a7，a8，a9}

R＝{r}

r＝{<a7,a2>，<a2,a1>，<a1,a6>，<a6,a8>，
<a8,a4>，<a4,a5>，<a5,a3>，<a3,a9>}

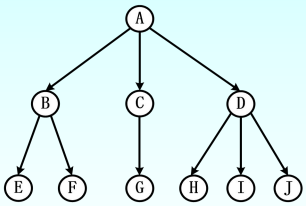
- 有序对<x，y>

  x指向y，称x是y的前驱，y是x的后继。

- 无序对(x，y)

  x，y之间有关系，但无顺序。

逻辑结构——树形结构



“一对多”

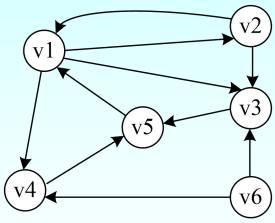
Tree＝（D，R），其中：

D＝{A，B，C，D，E，F，G，H，I，J}

R＝{r}

r＝{<A,B>，<A,C>，<A,D>，<B,E>，<B,F>，
<C,G>，<D,H>，<D,I>，<D,J>}

逻辑结构——图形结构



“多对多”

Graph＝（D，R），其中：

D＝{v1,v2,v3,v4,v5,v6}

R＝{r}

r＝{<v1,v2>，<v1,v3>，<v1,v4>，<v2,v1>，
<v2,v3>，<v3,v5>，<v4,v5>，<v5,v1>，
<v6,v3>，<v6,v4>}

逻辑结构的例题1

例1：以下二个用二元组表示的数据结构，画出它们对应的图形，并指出它们分别属于哪种逻辑结构。

（1）A＝（D，R），其中

D＝{a, b, c, d, e}

R＝{<a, b>, <a, c>, <b, d>, <c, e>}

（2）B＝（D，R），其中

D＝{a, b, c, d, e}

R＝{(a, b), (a, c), (b, d), (c, e) }

逻辑结构的例题2

例2：以下是用二元组表示的一个数据结构，画出它对应的图形，并指出它属于哪种逻辑结构。

B＝（D，R），其中

D＝{k1,k2,k3,k4,k5,k6}

R＝{r1,r2}

r1={<k3,k2>,<k3,k5>,<k2,k1>,<k5,k4>,<k5,k6>}

r2={<k1,k2>,<k2,k3>,<k3,k4>,<k4,k5>,<k5,k6>}

存储结构

➢ *存储结构的定义*：

又称物理结构，是逻辑结构在计算机存储器中的表现，不仅要存储数据元素本身，还要把数据元素之间的关系体现出来。

➢存储结构 { 顺序结构 / 链式结构 }

存储结构——顺序存储结构

➢ *顺序存储结构*：

用数据元素在存储器中的相对位置来表示元素之间的逻辑关系。

存储地址	存储状态
	:
s	元素1
s+L	元素2
:	:
s+(i-1)*L	元素i
:	:
s+(n-1)*L	元素n
:	:

如果已知Loc(元素1)=s，则

Loc(元素i)=s +(i-1)*L

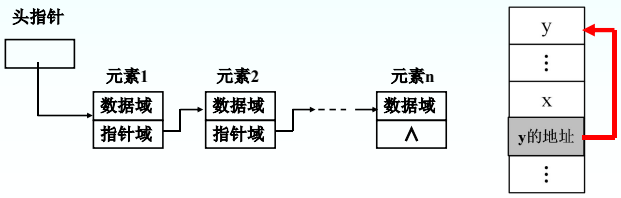
数据元素存放在连续存储单元

存储结构——链式结构

➢ *链式存储结构*：

结点（数据元素）之间的逻辑关系是由附加的指针项表示的。

➢每个结点由两个部分组成 { *数据域*—存放元素本身的数据 / *指针域*—存放与本数据元素相邻的元素的地址，即指针 }



存储结构的例题1

例1：一线性结构：a→b→c→d，且已知a的存储地址是120；设每个数据元素占2Bytes，指针占2Bytes。

解：

顺序存储结构

存储地址	存储状态
:	:
120	a
122	b
124	c
126	d
:	:

链式存储结构

存储地址	存储状态	
:	:	
72	b	76
76	c	152
:	:	
120	a	72
:	:	
152	d	^

存储结构的例题2

例2：某线性表在内存中的存储如下：head指向头结点，则线性表的逻辑结构为____。

A.L=(a,b,c,d,e,f)

B.L=(a,c,f,b,e,d)

C.L=(f,a,e,b,c,d)

D.L=(c,b,e,a,f,d)

Head->3FFCH		4000H
4000H	a	4008H
4004H	b	4010H
4008H	c	4014H
400CH	d	^
4010H	e	400CH
4014H	f	4004H

顺序与链式存储结构的比较

	顺序存储结构	链式存储结构
优点	存储结构简单	不要求连续存储空间，使用灵活 无需保留备用空间
缺点	要求连续的存储空间 必须按最大要求设计存储空间	结点指针要占用额外存储空间

逻辑结构与存储结构的关系

•算法的设计取决于选定的逻辑结构，而算法的实现依赖于采用的存储结构

•同一种逻辑结构可采用不同的存储结构

1.3抽象数据类型(Abstract Data Type,ADT)

- 什么是抽象数据类型？

  ADT是指一个数学模型以及定义在此数学模型上的一组操作。

- ADT的特点：
  - 降低了软件设计的复杂性；
  - 提高了程序的可读性和可维护性；
  - 程序的正确性容易保证。

抽象数据类型的表示

- ADT由元素、关系及操作3种要素来定义。抽象数据类型用三元组来表示：

  (D、R、P)

  其中：D是数据对象；R是D上的关系集；P是对D的基本操作集。

- 数据类型与抽象数据类型
  - 抽象数据类型需要通过高级编程语言中已经实现的数据类型（通常称之为固有数据类型）来实现；
  - 抽象数据类型的实现包括数据结构的实现和操作的实现；

ADT的说明形式

```
ADT 抽象数据类型名称
{
        数据对象：
                …
        数据关系：
                …
        操作集合：
                操作名1;
                        …
                        …
        操作名n;
}ADT抽象数据类型名称
```

ADT的例题

将矩形定义为抽象数据类型：

```
ADT RECtangle
{
        数据对象：D={lenght，width｜length和width是
                    矩形的长度和宽度}
        数据关系：R={（length，width）}
                  //矩形与length和width有关
   操作集合：
                  Circumference（r）;//求矩形r的周长
                  Area（r）；          //求矩形r的面积
}ADT RECtangle
```

用类C语言表示和实现ADT

- 类C语言就是类似C语言的语言。在数据结构中常用,在编程中有时也叫“伪代码”。其实就是为了让编程人员能够更专注对程序算法的思想分析，而不受语法的约束。所以一般分析问题，设计算法时“随意”写的算法代码（不考虑语法错误）就是类C语言。
- 类C语言只是在C语言编程过程中的一个思路的编写过程。
- 类C省略的是主函数，数据的输入输出，它主要通过函数的参数来完成数据的传递工作。
- 书P10-11 —— 类C语言

1.4 算法与算法分析



1.4.1 算法

1.4.2 算法设计的要求

1.4.3 算法的度量

1.4.4 算法分析

1.4.1 算法概述



➢ *算法*：解决一个问题的一系列计算步骤（或完成某一任务的操作步骤）。

➢ *特征*：

有穷性——每一条指令的执行次数必须是有限的，且在有穷时间内完成；

确定性——每一条指令的含义必须明确，无二义；

可行性——每一条指令都可通过已经实现的基本运算实现；

输 入——具有零个或多个输入；

输 出——至少产生一个输出。

1.4.2 算法设计的要求



设计一个“好”算法一般从4个方面进行：

- *正确性*——算法是否正确，是设计和评价一个算法的首要条件。
- *可读性*——算法易于阅读和理解。
- *健壮性*——算法对错误数据具有反应或处理的能力。
- *效率与低存储量要求*
  - ✓ *运行时间*——一个算法在计算机上运行所花费的时间；
  - ✓ *占用空间*——在计算存储器上所占用的存储空间，主要考虑在计算过程中临时占用的存储空间的大小。

1.4.3 算法的度量



➢算法的空间复杂度（占用空间）

算法在计算机内执行时所需要的存储空间，主要考虑在计算过程中临时占用的存储空间的大小。

算法1（需要额外的空间）

```
void swap (int &a, int &b)
{
    int temp;
    temp=a;
    a=b;
    b=temp;
}
```

算法2（不需要额外的空间）

```
void swap (int &a, int &b)
{
    a=a+b;
    b=a-b;
    a=a-b;
}
```

算法的时间复杂度（运行时间）



➢算法的时间复杂度：运行算法时所需要消耗的时间。

运行时间＝∑每条语句的执行时间

＝∑每条语句的执行次数

×该语句执行一次所需的时间

频度:记作f（n）

归一化，记为单位时间

时间复杂度：若解决一个问题的规模为n，则时间复杂度是n的函数，记作T（n）

T（n）＝∑f（n）

1.4.4 算法分析



（1）频度和时间复杂度的计算

（2）渐近时间复杂度和数学基础

（3）时间复杂度的几个法则

频度和时间复杂度的计算

例1：累加求和

Step	程序段	f(n)
1	s=0;	1
2	for (i=1;i<=n;i++)	n+1
3	s=s+A[i];	n
4	sum=s;	1

$T(n)=\sum f(n)=1+(n+1)+n+1=2n+3$

频度和时间复杂度的计算

例2：矩阵相乘C＝A×B（A、B、C分别为n×n阶矩阵）

Step	程序段	f(n)
1	for (i=1;i<=n;i++)	n+1
2	for (j=1;j<=n;j++)	n(n+1)
3	{ C[i][j]=0;	n^2
4	for (k=1;k<=n;k++)	$n^2(n+1)$
5	C[i][j]=C[i][j]+A[i][k]*B[k][j]; }	n^3

$T(n)=\sum f(n)= 2n^3+3n^2+2n+1$

渐进时间复杂度和数学基础

当问题的规模$n\to\infty$时，

时间复杂度——→渐进时间复杂度

（用T（n）的数量级表示）

数学基础：

设T（n）的一个辅助函数f（n），若n足够大时，有

$$\lim_{n\to\infty}\frac{T(n)}{f(n)}=\text{常数}\neq 0$$

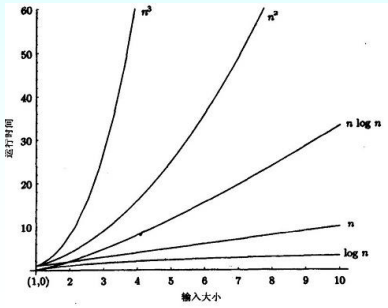
则称f(n)是T(n)的同数量级函数，或T(n)与f(n)同阶。

记作：T(n)=O(f(n))

常见的时间复杂度的增长率

$$O(1)\leqslant O(\log_2 n)\leqslant O(n)\leqslant O(n\log_2 n)\leqslant O(n^2)$$

$$\leqslant O(n^3)\leqslant\ldots\ldots\leqslant O(n^k)\leqslant O(2^n)\leqslant O(n!)$$



例题1、2的渐进时间复杂度

在例1中，T(n)=2n+3，取f(n)=n，得

$$\lim_{n\to\infty}\frac{T(n)}{f(n)}=\lim_{n\to\infty}\frac{2n+3}{n}=2$$

故，T(n) = O(f(n)) = O(n)

同理，在例2中，$T(n)=2n^3+3n^2+2n+1$，取$f(n)=n^3$，得

$T(n)=O(n^3)$

定理：当T(n)是n的多项式时，则f(n)为T(n)的最高次幂。

换言之，f(n)一般取算法中频度最大的语句频度。

时间复杂度的几个法则

法则1：赋值语句或读写语句的T(n)=O(1)。

例3：

```
temp=i;
i=j;        ⟹   T(n) = O(1)
j=temp;
```

时间复杂度的几个法则

法则2：T1(n)=O(f(n))、T2 (n)=O(g(n))分别是程序P1和P2的时间复杂度，则在执行P1后接着执行P2的总时间复杂度满足加法法则：

$$T(n)=T1(n)+T2(n)= O(\max (f(n), g(n)))$$

例4：

```
for (k=1;k<=n;k++)
    x=x+1;
for (i=1;i<=n;i++)
    for (j=1;j<=n;j++)
        y=y+1;
```

T1(n)=O(n)

T2(n)=O(n²)

⇒ T(n)=O(n²)

时间复杂度的几个法则

法则3：当有若干个循环语句时，T(n)是由嵌套层数最多的循环语句中最内层的f(n)决定。

例5：

```
x=1;
for (i=1;i<=n;i++)
    for (j=1;j<=i;j++)
        for ( k=1;k<=j;k++)
            x=x+1;    ……①
```

⇒

① 的频度：

$$f(n)=\sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=1}^{j}1=\frac{n^3+3n^2+2n}{6}$$

故：T(n)=O(n³)

时间复杂度的几个法则

法则4：T(n)不仅与n有关，还与它所处理的数据集的状态有关；在这种情况下，T(n)常用最坏时间复杂度表示。

例6：在数组A [n]中查找值为k的元素，若找到则返回其位置i(1≤i≤n)；否则返回0。

```
i=n;
while ((i>0) && (A[i] ≠k ))
    i=i-1;
return i;
```

⇒

最好情况 (A[n]=k)下：

T(n)=O(1)

最坏情况 (k不在 A中)下:

T(n)=O(n)

时间复杂度的几个法则

例7：while语句

```
i=1;
while (i<=n)
    i=i*3;  …… ①
```

⇒

设①的频度为f(n)，则有$3^{f(n)-1}\leqslant n$，得

$f(n)\leqslant \lfloor \log_3 n \rfloor +1$

取最大值，$f(n)=\lfloor \log_3 n \rfloor +1$，则

$T(n)=O(\log_3 n)$

时间复杂度的几个法则

法则5：

“if (条件B) S1 else S2”的运行时间

＝条件B的测试时间(通常定义为O(1))

＋分支S1、S2中运行时间大者

＝O(1)或分支S1、S2中运行时间大者

例8：

```
if （k>0）
{  for(i=0; i<n; i++)
       x=x+1;}
```

T(n)=O(n)

```
if （k>0）
   {  for(i=0; i<n; i++)
          x=x+1;}
else
{  for(i=0; i<n; i++)
          for(j=0; j<n; j++)
          x=x+1;}
```

T(n)=O(n²)

时间复杂度的几个法则

例9：递归函数

```
Fact(int n)
{
  if (n<=1) return 1;
  else  return (n*fact(n-1));
}
```

$$T(n)=\begin{cases} O(1) & n\leq 1 \\ T(n-1)+O(1) & n>1 \end{cases}$$

当n→∞，则

T(n)=O(1)+T(n-1)

=2*O(1)+T(n-2)

……

=i*O(1)+T(n-i)

……

=(n-1)*O(1)+T(1)

＝n*O(1)=O(n)

时间复杂度的几个法则

例10：递归函数

```
int Fib（int n）
{
  if (n<=1) return n
  else return (Fib(n-1)+Fib(n-2));
}
```

$$T(n)=\begin{cases}O(1) & n\le 1\\ T(n-1)+T(n-2)+O(1) & n>1\end{cases}$$

当n $\to\infty$时，

$$\begin{aligned}T(n) &= T(n-1)+T(n-2)+O(1)\\ &< 2[T(n-1)+O(1)]\\ &= 2[T(n-2)+T(n-3)+2O(1)]\\ &< 4[T(n-2)+O(1)]\\ &\vdots\\ &< 2^{i}[T(n-i)+O(1)]\\ &\vdots\\ &< 2^{n-1}[T(1)+O(1)]\\ &= 2^{n}O(1)\\ \therefore\quad T(n) &= O(2^{n})\end{aligned}$$

思考题

1. （1）在数据结构课程中，数据的逻辑结构、数据的存储结构及数据的运算之间存在着怎样的关系？

参考答案：

数据的逻辑结构反映数据元素之间的逻辑关系（即数据元素之间的关联方式或“邻接关系”）。

数据的存储结构是数据结构在计算机中的表示，包括数据元素的表示及其关系的表示。

数据的运算是对数据定义的一组操作，运算是定义在逻辑结构上的、和存储结构无关，而运算的实现则是依赖于存储结构。

思考题

（2）若逻辑结构相同但存储结构不同，则为不同的数据结构。这样的说法对吗？举例说明之。

参考答案：

逻辑结构相同但存储不同，可以是不同的数据结构。例如，线性表的逻辑结构属于线性结构，采用顺序存储结构为顺序表，而采用链式存储结构称为线性链表。

（3）在给定的逻辑结构及其存储表示上可以定义不同的运算集合，从而得到不同的数据结构。这样说法对吗？举例说明之。

参考答案：

对的。栈和队列的逻辑结构相同，其存储表示也可相同（顺序存储和链式存储），但由于其运算集合不同而成为不同的数据结构。

思考题

（4）评价各种不同数据结构的标准是什么？

参考答案：

数据结构的评价非常复杂，可以考虑两个方面：

一是所选数据结构是否准确、完整的刻划了问题的基本特征；

二是是否容易实现（如对数据分解是否恰当；逻辑结构的选择是否适合于运算的功能，是否有利于运算的实现；基本运算的选择是否恰当。）

思考题

2. 在编制管理通讯录的程序时，什么样的数据结构合适？为什么？

参考答案：

应从两方面进行讨论：

（1）如通讯录较少变动（如城市私人电话号码），主要用于查询，以顺序存储较方便，既能顺序查找也可随机查找；

（2）若通讯录经常有增删操作，用链式存储结构较为合适，将每个人的情况作为一个元素（即一个结点存放一个人），设姓名作关键字，链表安排成有序表，这样可提高查询速度。

运行时间的计算题

例1：

```
for ( i=2 ; i<= n-2 ; ++i )
    for ( j=n ; j >= i ; --j)
        { S ;}
```

求“S”语句的频度及整个程序段的算法复杂度

分析：i=2：执行 n-1 次

i=3：执行 n-2 次

…

i=n-2：执行 3 次

则：$f(n) = 3+4+5+\ldots+n-1 = (n-3)(n+2)/2$

$T(n) = O(n^2)$

或 $f(n) = \sum_{i=2}^{n-2}\sum_{j=n}^{i} 1$

运行时间的计算题

例2：
```
(1) for ( i=1; i<=n-1; ++i ) {
(2)     for ( j=n; j>=i+1; --j ) {
(3)         if ( A[j-1]>A[j] ) {
(4)             temp = A[j-1];
(5)             A[j-1]=A[j];
(6)             A[j]=temp;
            }
        }
    }
```

分析：

(4)~(6): O(1)

(3)~(6): O(1)

(2)~(6): O(1×(n-i))

= O(n-i)

(1)~(6):

$$O\left(\sum_{i=1}^{n-1}(n-i)\right)=O(n^2)$$

T(n)=O(n²)

运行时间的计算题

例3：
```
x = n ;  y = 0 ;
while ( x >= (y+1)(y+1)) do
@         y = y+1 ;
```

求@语句的频度及整个程序段的算法复杂度

分析：f(n)×f(n) <= n

$$\Rightarrow f(n)=\lfloor\sqrt{n}\rfloor$$

$$T(n)=O(\sqrt{n})$$

运行时间的计算题

例4：
```
w = 11 ;  k= 21 ;
while ( k > 10 ) do
@ if (w > 20)
    then {w=w-10 ; k--}
    else
     w=w+10;
```

求@语句的频度及整个程序段的算法复杂度

分析：对每一合法k值，@句执行2次

则，@句频度f(n)= (21-10)×2＝22

T(n)=O(1)

运行时间的计算题

例5：
```
x = 87 ;  y= 10 ;
while ( y > 0 )
{
 @ if (x> 100 )
    {x - = 10 ; y--}
    else
    x++;
}
```

求@语句的频度及整个程序段的算法复杂度

解：f(n)=15+11×9=114

T(n)=O(1)

运行时间的计算题

例6：
```
m=0; (n为偶数)
for (i=1; i≤n; i++)
{
    for (j=2*i; j≤n; j++)
    {
      @ m=m+1;
    }
}
```

求@语句的频度及整个程序段的算法复杂度

分析：i=1执行n-1次，i=2执行n-3次，i=n/2执行1次，因此 f(n)=1+3+5+…+(n-3)+(n-1),等差数列，首项为1，末项为(n-1)，项数为n/2

解：f (n)=n²/4

T(n)=O(n²)

运行时间的计算题

例7：
```
i=1;
j=0
while (i+j) ≤n
{
 if (i>j)
@  j=j+1;
 else
 i=i+1;
}
```

求@语句的频度及整个程序段的算法复杂度

分析：f(n)+f(n)<=n

$$f(n)=\left\lfloor\frac{n}{2}\right\rfloor,\quad T(n)=O(n)$$